ADAPT-G

Technical Project Brief & Implementation Roadmap

Adversarial Diffusion and Probabilistic Transformer-Graphs for Financial Fraud Detection

Purpose: Define the research problem, system design, implementation sequence, evaluation strategy, and paper structure.

Document purpose and status

This brief is the shared technical reference for understanding, implementing, evaluating, and writing up Adapt-G. It is intentionally both a research specification and a ticket-oriented build plan. The architecture is a research hypothesis, not a claim that the proposed combination will outperform existing methods. The project must establish that through controlled experiments.

- Working title: Adapt-G: Adversarial Diffusion and Probabilistic Transformer-Graphs for Financial Fraud Detection.
- Research type: empirical machine-learning research with a proposed model, controlled baselines, adversarial evaluation, and uncertainty analysis.
- Primary output: reproducible code, experimental results, a technical paper, and a clear account of limitations.
- Scope principle: establish a strong, leakage-safe baseline before adding the more expensive generator and hybrid model.
- Unified-model principle: components are first built, trained, and evaluated individually, but the final Adapt-G model is a single unified architecture that is trained as one unit, evaluated on its own, and compared against the baselines and against the standalone components it was assembled from (see "Unified-model requirement" in Section 5).

1. Project overview

Adapt-G investigates whether financial fraud detection can become more resilient to changing and deliberately deceptive fraud behaviour by combining (1) synthetic adversarial fraud generation, (2) temporal transaction modelling, (3) relational graph learning, and (4) calibrated uncertainty estimates. The intended setting is a digital-payment or banking transaction environment in which individual transactions have attributes, accounts and merchants interact over time, and fraud patterns evolve.

The key challenge is not merely to identify statistically unusual transactions. Fraud can be rare, coordinated across multiple entities, temporally structured, and intentionally shaped to resemble legitimate activity. Adapt-G therefore evaluates transactions in context: what happened, when it happened, how the entities are connected, and how reliable the model's prediction appears to be.

2. Problem statement and research questions

Conventional supervised fraud detectors can degrade when transaction distributions shift or attackers adapt to known decision boundaries. Static training data may not represent emerging patterns; transaction-only models may miss coordinated behaviour; and a point probability without meaningful uncertainty can encourage overconfident automated decisions.

Research questions

- RQ1: Does adding temporal and graph context improve fraud detection over strong tabular and temporal-only baselines?
- RQ2: Does adversarial synthetic-fraud training improve performance on held-out, realistically perturbed fraud patterns and temporal shifts?
- RQ3: Does the probabilistic output mechanism produce useful, calibrated uncertainty, particularly on difficult or shifted examples?
- RQ4: Which components contribute most to performance, robustness, and calibration, as measured by ablation studies?
- RQ5: What are the computational costs and practical trade-offs of the complete framework?
- RQ6: Does the unified Adapt-G model, trained end-to-end as a single unit, outperform both the external baselines and the standalone components (tabular, temporal, graph) from which it was assembled?

3. Aim and objectives

Overall aim

To design and empirically evaluate an adaptive, uncertainty-aware fraud detection framework that combines adversarial fraud synthesis with temporal and relational transaction modelling.

Specific objectives

- Build a reproducible, leakage-safe transaction-data pipeline with documented temporal splits and class-imbalance handling.
- Establish competitive baseline models and a clear evaluation protocol before introducing the full architecture.
- Represent transaction histories and entity relationships in a form that supports temporal and graph learning.
- Implement and validate a conditional diffusion generator for synthetic fraud, checking both realism and utility rather than assuming generated records are valid.
- Train a Transformer-GNN detector that combines temporal transaction context with graph neighbourhood information.
- Integrate the validated temporal, graph, fusion, and probabilistic components into a single unified Adapt-G model that is trained end-to-end as one unit, evaluated on its own, and compared against the external baselines and the standalone components built before the merge.
- Estimate predictive uncertainty and evaluate calibration, selective prediction, and error concentration on uncertain cases.
- Measure robustness under controlled adversarial perturbations and time-based distribution shifts without contaminating evaluation data.
- Run ablations, sensitivity checks, and efficiency tests to identify which components justify their complexity.
- Produce reproducible results, a limitations analysis, and a paper whose claims are directly supported by evidence.

4. Scope, assumptions, and non-goals

In scope

- Offline research on labelled transaction data, with time-aware evaluation.
- Transaction features and available entity links, such as account, merchant, device, or payment instrument identifiers.
- Synthetic-data generation for training augmentation and controlled robustness tests.
- Temporal and graph-based models, probabilistic prediction, calibration, and selective review analysis.
- Reproducible experiments and resource reporting.

Out of scope for the initial research

- Deploying a live production blocking system or making real financial decisions.
- Assuming generated fraud samples represent real future attacks without validation.
- Claiming formal security against all adaptive attackers.
- Using future information, post-outcome labels, or unavailable identifiers as model inputs.
- Building a complex diffusion-plus-graph architecture before confirming that simpler models and the data support it.

Important design choice: synthetic records must obey domain constraints and must not be presented as observed real-world fraud. If the available dataset lacks reliable entity identifiers or graph structure, the graph component must be redesigned, simulated only for a clearly labelled experiment, or removed; graph complexity should not be fabricated from arbitrary correlations.

5. Proposed system architecture

Component	Role	Inputs	Outputs / checks
Data and feature pipeline	Clean, validate, time-order, and split transactions.	Raw transactions, labels, timestamps, entity IDs.	Versioned feature tables; leakage audit; data-quality report.
Temporal encoder	Learn patterns across an entity's transaction history.	Ordered sequences and time gaps.	Contextual sequence embeddings.
Graph builder + GNN	Capture interactions and coordinated behaviour.	Entities and timestamped transaction edges.	Entity/transaction neighbourhood embeddings.
Conditional diffusion generator	Synthesize challenging fraud-like examples for training.	Training-fold features and class condition.	Synthetic candidates plus validity/utility report.
Probabilistic prediction head	Predict fraud risk and estimate uncertainty.	Fused temporal and graph representations.	Fraud score and uncertainty estimate.
Unified Adapt-G model	Single end-to-end architecture that integrates the temporal encoder, graph encoder, fusion layer, and probabilistic head; trained as one unit and evaluated on its own.	Transaction history sequence and graph neighbourhood, with the controlled real/synthetic training mixture.	One checkpoint producing fraud score and uncertainty; compared with baselines and pre-merge components.
Adversarial training controller	Schedule generation, detector updates, and evaluation.	Training metrics and fixed validation protocol.	Versioned training runs; no test-set feedback.
Evaluation layer	Compare accuracy, robustness, calibration, and cost.	Predictions, labels, perturbation suites.	Tables, curves, confidence intervals, and error analysis.

Unified-model requirement

Principle: Adapt-G is one unified architecture, not a collection of separate models. The components are first built, trained, and evaluated individually so that each one's contribution can be measured. The final Adapt-G model must then integrate them into a single model that can be trained as a unit, evaluated on its own, and compared against the external baselines and against the standalone components it was assembled from.

- **Two development phases.** Phase A (AG-006 to AG-009): train and evaluate the tabular, temporal, and graph components individually against the baselines; freeze their configurations, checkpoints, and predictions as reference points. Phase B (AG-010 onwards): merge the validated components into the unified Adapt-G model.
- **What "unified" means.** One model class and configuration containing the temporal encoder, graph encoder, fusion layer, and probabilistic head; one forward pass from transaction history and graph neighbourhood to fraud score and uncertainty; one training objective and optimiser, with gradients reaching every component; one training entry point and one resulting checkpoint.
- **Not acceptable as the headline Adapt-G model.** Stacking or averaging the outputs of separately trained models; feeding baseline predictions or scores into Adapt-G as inputs; training only a head on top of frozen pre-trained encoders; or any pipeline that needs separately trained components at inference time. Such systems may be reported, but only as clearly labelled ensemble or stacked baselines.
- **Pre-trained initialisation.** Initialising encoders from the standalone component weights is allowed only as a declared variant, with all parameters unfrozen and trained jointly. The headline result must be reproducible by training the unified architecture end-to-end, and both variants are reported if warm-starting is used.
- **Standalone evaluation.** The unified model is evaluated on its own: one checkpoint, one inference path, the standard evaluation harness (AG-005), validation-selected thresholds, and the same

clean, shifted, stress, calibration, uncertainty, and efficiency evaluations applied to every other model.

- **Three-tier comparison.** Every main results table includes (1) the simple, tabular, and other external baselines; (2) the pre-merge components (tabular, temporal-only, graph-only) exactly as built and evaluated in Phase A; and (3) the unified Adapt-G model and its unified ablation variants. All tiers use identical splits, features, threshold-selection rules, seeds or time windows, and a comparable tuning budget, and parameter counts are reported so that gains are not simply attributed to extra capacity.
- **Role of the diffusion generator.** The conditional generator belongs to the Adapt-G training framework: it is trained, scheduled, and updated within the adaptive loop (Section 6.7). It is not on the inference path, so at inference the unified detector runs on its own, and that is the artefact evaluated and benchmarked for latency.
- **Honest outcome.** If the unified model does not beat its best individual component or baseline, that is reported as a finding and handled by the go/no-go rules in Section 14, after first ruling out joint-training issues such as loss imbalance, mismatched learning rates between components, or an insufficient tuning budget.

High-level data flow

Raw transactions → quality checks and temporal split → leakage-safe features → temporal sequences + entity graph → baseline and standalone component detectors (tabular, temporal, graph), evaluated individually and frozen as reference points → conditional generator trained only on permitted training data → synthetic sample validation → unified Adapt-G model (temporal encoder + graph encoder + fusion + probabilistic head, trained end-to-end as one unit) → standard, shifted, and adversarial evaluation of the unified model alongside the baselines and pre-merge components → ablation and error analysis.

6. Technical approach

6.1 Data representation and leakage control

- Define the prediction unit explicitly: for example, one transaction at authorisation time. Specify the exact point at which a prediction must be made.
- Separate identifiers used to construct relationships from raw identifiers used as model features. Avoid memorising account IDs unless that is an explicit, justified experimental choice.
- Create chronological train, validation, and test splits. Fit encoders, scalers, imputation rules, feature selection, and generators on training data only.
- Use only information available at prediction time. Remove post-transaction outcomes, chargeback results that arrive later, future aggregates, and target-derived fields.
- Build temporal aggregates with rolling or expanding windows that end before the target transaction.
- Document label delay, duplicate handling, missingness, class prevalence, and whether the dataset contains repeated entities across splits.

6.2 Temporal modelling

Construct a bounded sequence of prior transactions for each relevant entity (such as an account or card token), sorted by event time. Encode transaction attributes, time gaps, and optionally positional/time features. The Transformer should produce a representation for the current transaction using only prior context. Compare it with a non-sequential baseline and a simpler sequence model where feasible.

6.3 Graph construction and relational learning

Represent available entities as nodes and observed transactions as timestamped, attributed edges, or use a transaction-centric heterogeneous graph if that better fits the dataset. Specify node types, edge

direction, edge attributes, temporal cut-off, neighbourhood sampling, and treatment of unseen entities. A graph snapshot for a transaction must contain only edges observable before or at the prediction point. Evaluate whether graph features add value beyond temporal and tabular features.

6.4 Conditional diffusion fraud generator

Train a conditional diffusion model to produce candidate minority-class examples or latent representations conditioned on training-fold fraud labels and selected context. The generator should be trained only on training data. Its purpose is to create challenging, plausible training examples, not to certify future attack behaviour.

- Choose a representation compatible with mixed tabular data. Consider latent-space diffusion or a mixed-type diffusion method if direct diffusion over categorical and continuous features is unsuitable.
- Define conditioning carefully; a fraud label alone may not capture relevant fraud subtypes or temporal context.
- If using an adversarial objective, define the game explicitly: the generator seeks examples that reduce detector confidence or increase detector loss, while constraints preserve domain validity and the fraud condition.
- Prevent the generator from winning by producing invalid records, label flips, impossible feature combinations, or near-duplicates of training examples.
- Assess synthetic quality with feature-wise checks, domain constraints, nearest-neighbour/privacy checks where appropriate, downstream utility, and performance on a separately defined stress set.
- Keep the test set isolated. Never select generator settings based on test-set scores.

6.5 Transformer-GNN fusion

Encode the current transaction's history with a Transformer and its relevant graph neighbourhood with a GNN. Fuse the resulting representations through concatenation plus a projection layer, gated fusion, or another pre-specified simple mechanism. Start with the simplest fusion design and add complexity only if experiments justify it. The temporal encoder, graph encoder, fusion layer, and head must form one unified model trained end-to-end with a single objective, so that gradients reach every component. The fused model is not assembled by stacking or averaging separately trained models (see "Unified-model requirement" in Section 5).

6.6 Probabilistic output and uncertainty

A practical first implementation is Monte Carlo Dropout: perform multiple stochastic forward passes at inference and summarise the mean fraud score and predictive variability. The dropout layers and probabilistic head are part of the unified model's architecture, so uncertainty comes from the same model that produces the fraud score. Compare this with a deterministic head and, if resources permit, an ensemble. Distinguish uncertainty estimates from calibrated fraud probabilities: they are related but not interchangeable.

- Report discrimination and operating-point metrics separately from calibration.
- Evaluate calibration with reliability diagrams, Brier score and/or log loss, and expected calibration error with clearly stated binning.
- Evaluate whether uncertainty identifies errors or distribution-shifted examples, and whether abstaining on high-uncertainty cases improves the retained predictions.
- State the limitations of the chosen uncertainty proxy; Monte Carlo Dropout is not a guarantee of well-calibrated epistemic uncertainty.

6.7 Adaptive training loop

- Train the baseline and standalone component detectors (tabular, temporal, graph), keep their results as frozen reference points, and freeze the validation protocol.
- Generate candidate synthetic examples from the training fold only.
- Filter candidates through validity and quality checks; record rejection reasons.

- Train or fine-tune the unified Adapt-G model end-to-end, with all components trainable, using a controlled real/synthetic mixture.
- Evaluate on validation data and a fixed, separate robustness suite.
- Adjust generator difficulty or mixture weight using pre-defined validation criteria, not test performance.
- Save each generator/detector version, configuration, random seed, data version, and metric set.
- Stop or roll back if synthetic augmentation harms clean performance, calibration, or data validity.

7. Evaluation and evidence plan

7.1 Datasets and split protocol

Select a publicly usable, documented fraud dataset or an approved internal dataset. The final dataset choice must be based on label quality, temporal information, feature availability, entity identifiers, licensing/privacy constraints, and whether it supports the proposed graph representation. If no single dataset supports every component, define a primary dataset and clearly labelled supplementary experiments.

Use chronological splits whenever timestamps permit. Preserve a final untouched test period. Where multiple random seeds or repeated time windows are feasible, report variability rather than a single favourable run. Do not use synthetic samples in validation or test sets as substitutes for real held-out examples.

7.2 Baselines

- Simple reference: majority-class predictor and a basic rule/threshold reference where appropriate.
- Tabular supervised baselines: Logistic Regression and a strong tree-based model such as XGBoost, LightGBM, or CatBoost, depending on feature compatibility.
- Temporal baseline: a sequence model or Transformer without graph context.
- Graph baseline: GNN without Transformer history encoding, if graph data are meaningful.
- Pre-merge component references: the tabular, temporal, and graph models exactly as built and evaluated in AG-006, AG-007, and AG-009, kept frozen (checkpoints, configurations, predictions) and re-evaluated on the final test period under the same protocol as the unified model.
- Hybrid ablations (each a unified model trained end-to-end under the same protocol): Transformer-GNN without synthetic augmentation; with ordinary class weighting/oversampling; with diffusion augmentation; deterministic versus probabilistic head.
- Primary comparison: the unified Adapt-G model against every external baseline and every pre-merge component above, on identical splits, features, thresholds, and tuning budget, with parameter counts reported.
- Where feasible, compare with a standard anomaly-detection baseline, but do not treat anomaly detection as a substitute for supervised fraud classification.

7.3 Metrics

Evaluation dimension	Recommended measures	Interpretation / caution
Ranking / detection	PR-AUC (average precision), ROC-AUC	PR-AUC is especially informative under heavy class imbalance; report definitions.
Operational errors	Precision, recall, F1, false-positive rate, false negatives at chosen thresholds	Thresholds must be selected on validation data and tied to a plausible review/cost setting.
Cost / workload	Precision at a fixed alert budget; recall at a fixed false-positive rate; estimated cost where assumptions are stated	Avoid claiming business impact without defensible cost assumptions.
Calibration	Brier score, log loss, reliability plot, ECE	ECE depends on binning; report methodology and avoid relying on one calibration metric.

Uncertainty usefulness	Error-detection AUROC/AUPRC, risk-coverage curve, selective risk	Tests whether abstaining on uncertain cases is useful.
Robustness	Metric degradation under each defined perturbation/shift; worst-case or grouped results	Perturbations must be plausible and label-preserving where required.
Efficiency	Training time, inference latency, memory, parameter count, generator overhead	State hardware, batch size, and measurement protocol.

7.4 Robustness test suite

- Temporal shift: train on earlier periods and test on a later period.
- Feature perturbations: controlled changes to selected amounts, timing, channel, or other mutable fields within plausible bounds.
- Context/relational stress: evaluate sparsely connected, newly appearing, or structurally altered entity neighbourhoods when the dataset permits.
- Generator-based hard examples: use a held-out generation/evaluation protocol that avoids training-test leakage.
- Missingness/noise: simulate bounded feature missingness or noise as a separate stress test.
- Label-preservation audit: document which transformations are expected to preserve fraud status and how that assumption is validated.

Do not equate arbitrary feature perturbations with real adversarial attacks. Describe the threat model: attacker knowledge, capabilities, mutable fields, constraints, and objective. Report clean performance alongside robustness; a model that becomes robust only by producing excessive false positives is not a successful result.

8. Sequential implementation roadmap (ticket-based)

Implement in the order below. Each ticket should produce a reviewable artifact and acceptance evidence. Do not begin the next major modelling stage until its dependencies and acceptance criteria are satisfied. Ticket names are suggested IDs and can be moved into GitHub Issues, Jira, Linear, or another tracker.

AG-001 — Research specification and threat model

Work to do: Write a concise protocol defining the prediction unit, fraud label, attacker capabilities, mutable features, allowed knowledge, target deployment context, and success criteria. Identify what counts as a realistic adversarial example and what is out of scope.

Acceptance criteria: Approved specification; explicit research questions; threat-model table; no undefined claims such as 'robust to all attacks'.

AG-002 — Dataset selection and data audit

Work to do: Select candidate dataset(s), review licence/privacy conditions, inspect schema, timestamps, label semantics, class prevalence, missingness, duplicates, entity identifiers, and temporal coverage. Determine whether graph construction is genuinely supported.

Acceptance criteria: Dataset decision record; data dictionary; data-quality report; documented graph feasibility; reproducible raw-data loading.

AG-003 — Repository, configuration, and reproducibility scaffold

Work to do: Create a clean repository structure, environment lock/dependency file, configuration system, seeded runs, structured logging, experiment output conventions, and tests. Keep data and secrets out of version control.

Acceptance criteria: Fresh environment can run a smoke test; configuration and seeds recorded; CI or local test command documented.

AG-004 — Leakage-safe preprocessing and temporal splits

Work to do: Implement schema validation, train/validation/test chronology, missing-value handling, feature transformations, training-only fitting, and time-safe rolling aggregates. Document any entity overlap across periods.

Acceptance criteria: Split report and leakage checklist; deterministic preprocessing; unit tests prove no future data enter features.

AG-005 — Evaluation harness and metrics

Work to do: Implement metric calculation, threshold selection on validation only, confidence intervals or seed variability where practical, metric logging, and standard report generation.

Acceptance criteria: Tested metrics module; standard evaluation command; results always include PR-AUC and threshold-dependent error measures.

AG-006 — Strong tabular baselines

Work to do: Train simple and strong supervised baselines using identical splits and available features. Tune only on validation data; establish reproducible reference performance and error analysis.

Acceptance criteria: Baseline table; saved configurations/predictions; documented threshold; repeatable run; results frozen as the tabular reference for the AG-010 unified-model comparison.

AG-007 — Temporal sequence dataset and baseline

Work to do: Construct ordered transaction histories and time features using only information available at prediction time. Train a temporal model and compare against tabular baselines.

Acceptance criteria: Sequence builder tests; example sequences manually audited; temporal baseline results; latency and memory recorded; checkpoint and predictions frozen as the temporal reference for the AG-010 unified-model comparison.

AG-008 — Temporal graph construction

Work to do: Define node/edge types, direction, edge attributes, temporal cut-off, sampling strategy, cold-start handling, and graph snapshot creation. Verify each target example cannot access future edges.

Acceptance criteria: Graph schema diagram; graph statistics; leakage tests; sampled graph examples reviewed.

AG-009 — GNN-only baseline

Work to do: Train a graph detector without the Transformer to test whether graph context adds signal. Compare with tabular and temporal models and analyse performance by connectivity.

Acceptance criteria: GNN baseline results; connected versus sparse/new-entity analysis; graph cost report; checkpoint and predictions frozen as the graph reference for the AG-010 unified-model comparison.

AG-010 — Unified Adapt-G model (Transformer-GNN fusion detector)

Work to do: Assemble the temporal encoder, graph encoder, simple fusion layer, and classification head into a single unified model with one forward pass and one training objective, trained jointly end-to-end (all parameters trainable) from a single entry point. Evaluate it on its own with the standard harness, and compare it against the external baselines and the standalone pre-merge components from AG-006, AG-007, and AG-009 under identical splits, features, thresholds, seeds, and tuning budget.

Acceptance criteria: One unified model class and configuration; a single command trains all components jointly; tests confirm gradients reach every component; one checkpoint evaluated standalone with no dependence on separately trained components at inference; three-tier comparison

table (baselines, pre-merge components, unified model) with parameter counts and tuning budgets; ablation table for tabular, temporal, graph, and fused variants; any warm-started variant declared and reported next to the end-to-end result.

AG-011 — Uncertainty estimation and calibration

Work to do: Add Monte Carlo Dropout or another justified probabilistic mechanism to the unified Adapt-G model. Measure calibration, error detection, risk-coverage, and the effects of thresholding/abstention.

Acceptance criteria: Uncertainty outputs reproducible; calibration plots and metrics; deterministic-head comparison; limitations documented.

AG-012 — Synthetic fraud generator feasibility study

Work to do: Before full adversarial training, test a suitable generator on training-fold data only. Check mixed-type feature validity, memorisation, privacy risk, diversity, and downstream utility. Compare against no augmentation and simple class reweighting/oversampling.

Acceptance criteria: Synthetic-data quality report; validity constraints; nearest-neighbour/memorisation checks where appropriate; utility comparison; go/no-go decision.

AG-013 — Conditional diffusion implementation

Work to do: Implement the selected conditional diffusion approach and training objective. Define conditioning variables, continuous/categorical handling, sampling settings, and generated-sample filtering.

Acceptance criteria: Training and sampling reproducible; generated data pass documented validation; generator checkpoints and configuration saved.

AG-014 — Adversarial generator-detector interaction

Work to do: Define the generator's adversarial objective and constraints. Alternate generator and unified-model (detector) updates in a controlled schedule, avoiding test-set feedback and collapse to invalid or trivial samples.

Acceptance criteria: Stable training logs; attack/augmentation examples audited; generator difficulty and mixture weight tracked; rollback criteria implemented.

AG-015 — Robustness and distribution-shift suite

Work to do: Implement time-shift tests and a documented set of plausible perturbations. Ensure each transformation's constraints and label-preservation assumptions are explicit. Evaluate clean and stressed performance.

Acceptance criteria: Versioned stress suite; threat-model coverage table; clean-versus-stress results; no test leakage.

AG-016 — Ablations and sensitivity analysis

Work to do: Remove or replace one component at a time: no generator, ordinary oversampling, no graph, no temporal encoder, deterministic head, alternative fusion, and different synthetic-data ratios. Vary key settings within a planned budget. Each ablated variant is retrained end-to-end as its own unified model under the same protocol, rather than obtained by disabling parts of the trained full model.

Acceptance criteria: Ablation and sensitivity tables; evidence for each claimed contribution; negative results retained.

AG-017 — Statistical analysis and error investigation

Work to do: Repeat key experiments across seeds or time windows where feasible. Inspect false positives/negatives, fraud subgroups, calibration failures, generator artefacts, and subgroup performance where lawful and appropriate.

Acceptance criteria: Uncertainty intervals or variability report; error taxonomy; limitations and failure cases documented.

AG-018 — Efficiency and practical feasibility

Work to do: Measure training cost, inference latency, memory, model size, and generator overhead on declared hardware. Consider whether generation can be offline during training rather than on the inference path.

Acceptance criteria: Hardware/protocol description; efficiency table; deployment implications and bottlenecks stated.

AG-019 — Paper figures, tables, and reproducibility package

Work to do: Generate plots and tables from logged results, prepare architecture and training-loop diagrams, freeze experiment manifests, and document how to reproduce primary results.

Acceptance criteria: Every paper result traceable to a run; figures generated by scripts; setup and reproduction guide complete.

AG-020 — Paper drafting and final claim audit

Work to do: Write the paper against the agreed structure. Tie every claim to a result, distinguish measured findings from hypotheses, report negative outcomes and limitations, and audit citations and data permissions.

Acceptance criteria: Complete manuscript; claim-to-evidence checklist; reproducibility/ethics statements; final review completed.

9. Suggested repository structure

```
adapt-g/
├── README.md
├── pyproject.toml                # or requirements/lock files
├── configs/
│   ├── data/
│   ├── models/
│   ├── training/
│   └── evaluation/
├── docs/
│   ├── research_spec.md
│   ├── threat_model.md
│   ├── data_dictionary.md
│   └── experiment_protocol.md
├── src/adapt_g/
│   ├── data/                     # ingestion, validation, splits
│   ├── features/                 # leakage-safe transformations
│   ├── graph/                    # graph building and sampling
│   ├── models/
│   │   ├── baselines/
│   │   ├── temporal/
│   │   ├── gnn/
│   │   ├── diffusion/
│   │   └── hybrid/              # unified Adapt-G model (one end-to-end module)
│   ├── uncertainty/              # MC Dropout, calibration
│   ├── training/                 # loops, checkpoints, schedules
│   ├── evaluation/               # metrics, shifts, adversarial suite
│   └── utils/
```

```
├── tests/
│   ├── unit/
│   ├── integration/
│   └── leakage/
├── scripts/
│   ├── prepare_data.py
│   ├── train_baseline.py         # standalone component baselines
│   ├── train_detector.py         # trains unified Adapt-G end-to-end
│   ├── train_generator.py
│   ├── run_evaluation.py         # baselines, components, unified model
│   └── make_paper_figures.py
├── experiments/                   # manifests and lightweight summaries
└── paper/
    ├── main.tex
    ├── references.bib
    ├── figures/
    └── tables/
```

10. Proposed paper structure

Title and abstract — State the problem, proposed approach, evaluation setting, main quantitative results, and limitations. Do not write results into the abstract before they are measured.

1. Introduction — Explain adaptive financial fraud, why temporal and relational context matter, why adversarial evaluation and uncertainty matter, contributions, and research questions.

2. Problem formulation and threat model — Define transaction-level prediction, data available at decision time, attacker capabilities, realistic constraints, labels, and evaluation objectives.

3. Related work — Cover financial fraud detection, temporal/sequence models, graph-based fraud detection, synthetic tabular data/diffusion, adversarial ML, uncertainty estimation/calibration, and concept drift.

4. Methodology: Adapt-G — Describe data representation, temporal encoder, graph construction/GNN, conditional diffusion generator, adversarial objective, fusion mechanism, probabilistic head, how these are integrated into one unified model, and the end-to-end training algorithm.

5. Experimental setup — Datasets, splits, preprocessing, baselines, hyperparameter selection, metrics, threat models, hardware, random seeds, and implementation details.

6. Main results — Compare the unified Adapt-G model against the external baselines and the standalone pre-merge components on clean and time-shifted evaluation; report ranking, operational, calibration, and efficiency metrics.

7. Robustness and uncertainty analysis — Report adversarial/stress tests, calibration, risk-coverage, error analysis, and where uncertainty is or is not useful.

8. Ablation and sensitivity studies — Quantify the contribution of graph context, temporal context, diffusion augmentation, adversarial training, uncertainty estimation, and key design choices.

9. Discussion — Interpret findings, explain trade-offs, discuss practical implications, and distinguish supported conclusions from hypotheses.

10. Limitations, ethics, and responsible use — Discuss dataset representativeness, label delay, synthetic-data artefacts, privacy/memorisation risk, fairness concerns, false-positive harms, and attacker-model limits.

11. Conclusion — Answer the research questions based on evidence and identify specific future work.

Appendix / supplementary material — Include detailed hyperparameters, extra results, feature definitions, pseudo-code, data statements, and reproduction instructions.

11. Expected deliverables

- Research specification and explicit threat model.
- Dataset and leakage audit, with documented preprocessing and temporal splits.
- Reproducible baseline suite and evaluation harness.
- Temporal, graph, and fused detector implementations.
- A unified Adapt-G model that can be trained end-to-end from a single entry point and evaluated on its own, with a results table comparing it against the baselines and the pre-merge components.
- Conditional diffusion generator feasibility study and, if justified, adversarial training implementation.
- Robustness, calibration, ablation, error-analysis, and efficiency reports.
- A paper-ready set of tables, figures, and a manuscript.
- Reproduction documentation with environment, configurations, seeds, and run commands.

12. Risks, mitigations, and decision gates

Risk	Why it matters	Mitigation / decision gate
No reliable graph identifiers	A GNN may add no valid relational information.	Assess at AG-002/008; simplify or drop graph component if links are absent or unreliable.
Temporal leakage	Inflated scores invalidate the research.	Chronological splits, prediction-time audit, automated leakage tests, and independent review.
Synthetic samples are unrealistic	Generator may teach artefacts rather than fraud patterns.	Validity constraints, nearest-neighbour checks, downstream utility, stress tests, and comparison with simple augmentation.
Adversarial objective destabilises training	Generator may collapse or produce invalid samples.	Start with generator feasibility; constrain updates, log samples, use staged training and rollback.
Class imbalance hides failures	Accuracy can appear high while fraud recall is poor.	Prioritise PR-AUC and operational metrics; report class prevalence and thresholds.
Uncertainty is poorly calibrated	A confidence score may be misleading.	Evaluate calibration and selective risk; compare with deterministic and ensemble alternatives.
Excessive architecture complexity	Hard to attribute improvements or reproduce findings.	Baseline-first workflow, ablations, staged go/no-go decisions.
Final model is a pipeline of separately trained parts	Results would not evaluate Adapt-G as one architecture, and gains could come from stacking rather than joint modelling.	Unified-model requirement (Section 5); AG-010 acceptance criteria; single-checkpoint evaluation; tests that gradients reach every component.
Unified model does not beat its own components	Joint training can be harder to optimise than component-wise training, and extra capacity can mask this.	Compare against the frozen pre-merge components on an identical protocol; check loss balancing and tuning budget; report the outcome and apply the Section 14 go/no-go rule.
Data/privacy restrictions	Entity data may be sensitive or restricted.	Use approved access, minimise identifiers, document retention/access, avoid publishing raw sensitive records.

13. Research integrity and implementation rules

- No test-set tuning. The final test set is used only for final evaluation after design decisions are frozen.

- Every experiment must log the dataset version, split boundaries, feature configuration, model configuration, random seed, code version, and metrics.
- Report all primary metrics and meaningful negative results; do not select only the best seed or favourable attack.
- Synthetic records must be labelled as synthetic in the pipeline and never mixed into the held-out real test set.
- State whether attack examples are generated online during training, pre-generated offline, or used only for evaluation.
- Use confidence intervals or repeated-run variability for central claims when computationally feasible.
- Avoid claiming novelty solely because several known components are combined. The contribution must be a well-defined method and/or new empirical evidence supported by comparisons.
- The headline Adapt-G result must come from the unified model trained and evaluated as a single unit. Results for the baselines, the pre-merge components, and the unified model must use the same splits and protocol and must not be mixed or substituted for one another.
- Document ethical risks, false-positive consequences, dataset licensing, and any limitations in population or geography.

14. Recommended execution strategy

The highest-risk and most informative early decisions are data suitability, leakage control, baseline strength, and graph feasibility. The recommended sequence is therefore: specify the threat model → audit data → build the reproducible pipeline → establish strong baselines → validate temporal and graph signals → merge the validated components into the unified Adapt-G model, train it end-to-end, and compare it with the baselines and its own pre-merge components, adding uncertainty evaluation → test whether diffusion augmentation adds value → implement the adversarial loop only if the generator feasibility study passes → run robustness, ablation, and efficiency experiments → write the paper.

A useful go/no-go rule is to retain each added component only if it provides a reproducible improvement on a pre-defined research objective, or reveals a clearly justified trade-off. If diffusion does not outperform simpler augmentation, that is an important result—not a reason to conceal the comparison. If the GNN does not add value, report the finding and reconsider the scope rather than forcing the original architecture. Likewise, the unified model must be compared against its own best pre-merge component; if it does not improve on it, report that and diagnose the joint training before drawing conclusions.

15. Definition of success

Adapt-G is successful as a research project if it produces a rigorous, reproducible answer to whether adversarially generated training examples, temporal-plus-graph context, and uncertainty estimation, combined in a single unified model that is trained as one unit and compared fairly with the baselines and its own pre-merge components, improve fraud detection under realistic distribution shifts and attacker constraints. Success is not defined by using every proposed component; it is defined by credible evidence, sound comparisons, transparent limitations, and a useful technical contribution.